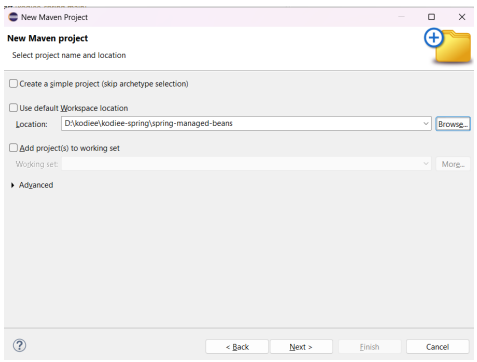
Creating and Spring-Managing Beans Using Annotations (Annotation Type)

Project: spring-managed-beans-using-annotation-type

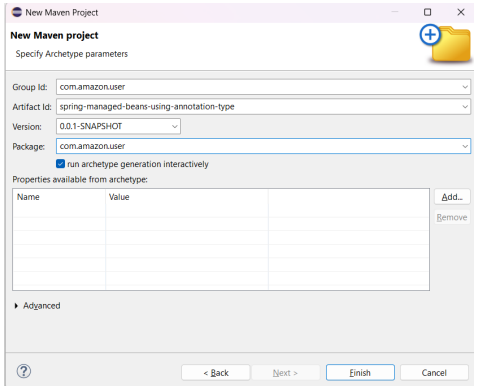
Step 1: Create the Maven project in Eclipse

- File → New → Maven Project → set the workspace location path → Next



Workspace location

- Set Catalog to Internal and select the archetype → Next
- Set Group Id (`com.amazon.user`), Artifact Id (`spring-managed-beans-using-annotation-type`), Version (`0.0.1-SNAPSHOT`) and Package (`com.amazon.user`) → Finish



Archetype parameters

- Type Y in the Console when prompted

Step 2: Add the Spring Context dependency

Open `pom.xml` and add the `spring-context` dependency from https://mvnrepository.com/artifact/org.springframework/spring-context. Always add one version below the latest.

pom.xml

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.19</version>
    <scope>compile</scope>
</dependency>
```

Step 3: Create the Employee class

Create the class `Employee` in package `com.amazon.employee` and mark it with `@Component`. We can inject the values in two ways.

Approach 1: Field Injection

```
package com.amazon.employee;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class Employee {

    @Value("101")
    private int id;

    @Value("Alpha")
    private String name;

    @Value("PREMIUM")
    private String band;

    public void displayInfo() {
        System.out.println("id=" + id + ", name=" + name + ", band=" + band);
    }
}
```

Approach 2: Constructor Injection

```
package com.amazon.employee;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class Employee {

    private int id;
    private String name;
    private String band;

    public Employee(
        @Value("101") int id,
        @Value("Alpha") String name,
        @Value("PREMIUM") String band
    ) {
        super();
        this.id = id;
        this.name = name;
        this.band = band;
    }

    public void displayInfo() {
        System.out.println("id=" + id + ", name=" + name + ", band=" + band);
    }
}
```

Note: Only one `Employee` class can exist at a time. Replace the class when switching between the two approaches.

Step 4: Create the Spring configuration class

Create the class `SpringConfig` in package `com.amazon.config`. Annotate it with `@Configuration` and `@ComponentScan(basePackages = "com.amazon")` so that Spring scans the package for `@Component` classes.

SpringConfig.java

```
package com.amazon.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "com.amazon")
public class SpringConfig {

}
```

Step 5: Load the bean in App.java

In `App.java` (package `com.amazon.user`), create the Spring container using `AnnotationConfigApplicationContext` and pass `SpringConfig.class` to it. Then get the `Employee` bean from the container by its type and call `displayInfo()`.

App.java

```
package com.amazon.user;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import com.amazon.config.SpringConfig;
import com.amazon.employee.Employee;

public class App
{
    public static void main(String[] args)
    {
        System.out.println("Creating Spring beans using Annotation Type");
        ApplicationContext context = new AnnotationConfigApplicationContext(SpringConfig.class);
        Employee employee = context.getBean(Employee.class);

        employee.displayInfo();
    }
}
```

Step 6: Run the application and check the output

Run `App.java` as a Java Application. The console prints the Employee bean created by Spring. The output is the same for both approaches.

Output

```
Creating Spring beans using Annotation Type
id=101, name=Alpha, band=PREMIUM
```